

מסמך ארכיטקטוני למבצע "רשת הכתבים"

1	מסמך ארכיטקטוני למבצע "רשת הכתבים"
1	תוכן עניינים
1	1. הקדמה
2	2. סקירת שירותים (Overview)
2	3. תרשים זרימה לוגי של המערכת
2	4. שלד מחלקות לפי שירות
3	Data Loader
3	Ingestion Service
3	Clean Service
4	Analytics Service
4	Elastic Indexer
4	Query API
5	Dashboard
5	5. Logging – ארכיטקטורה והטמעה
6	6. פירוט מחלקות – Constructor ומתודות
6	Data Loader
6	DataLoaderConfig
6	ImageDownloader
6	DownloadOrchestrator
6	Ingestion Service
6	IngestionConfig
7	OCREngine
7	GridFSStorage
7	KafkaPublisher
7	IngestionOrchestrator
7	Clean Service
7	CleanConfig
7	KafkaConsumer
8	TextCleaner
8	KafkaPublisher
8	CleanOrchestrator
8	Analytics Service
8	AnalyticsConfig
8	KafkaConsumer
8	TextAnalyzer
8	KafkaPublisher

9	AnalyticsOrchestrator
9	Elastic Indexer
9	IndexerConfig
9	KafkaConsumer
9	ElasticsearchClient
9	IndexOrchestrator
9	Query API
9	APICDashboardApponfig
10	SearchService
10	Routes / Controllers
10	Dashboard
10	

תוכן עניינים

1. הקדמה
2. סקירת שירותים (Overview)
3. תרשים זרימה לוגי של המערכת
4. שלד מחלקות לפי שירות
5. הרחבות עתידיות (לקריאה מעמיקה)

1. הקדמה

מסמך זה נועד לתת תמונה ארכיטקטונית ברורה וקצרה של מערכת "רשת הכתבים".

מטרתו:

- להציג את מבנה המערכת מלמעלה.
- להסביר אילו שירותים קיימים ומה תפקידו של כל אחד.
- להבהיר מה עליכם לממש כדי להתחיל לעבוד.

2. סקירת שירותים (Overview)

להלן רשימת השירותים במערכת ותפקידם הכללי:

Data Loader

מכין את סביבת העבודה על ידי טעינת מאגר התמונות לתיקייה מקומית (לדוגמה מתוך קובץ ZIP). אינו מתקשר עם Kafka או מסדי נתונים.

Ingestion Service

סורק את תיקיית התמונות, מבצע OCR, מחלץ מטא-דאטה בסיסי מהקובץ (כגון גודל, ממדים ופורמט), שולח את הקובץ הבינארי בלבד ל-MongoDB Loader דרך REST, ומפרסם אירוע RAW ל-Kafka הכולל את ה-image_id, הטקסט שחולץ והמטא-דאטה.

MongoDB Loader

מספק שכבת REST לקבלת קובצי תמונות בבקשת POST, ושומר את הקובץ הבינארי בלבד ב-MongoDB באמצעות GridFS. אינו שומר מטא-דאטה ואינו מכיר את ה-Pipeline האירועי.

Clean Service

צורך אירוע RAW, מנקה סימני פיסוק ותווים מיותרים מהטקסט ומפרסם אירוע CLEAN.

Analytics Service

צורך אירוע CLEAN, מחשב מדדים בסיסיים על הטקסט (כמות מילים, אורך, תבניות וכו') ומפרסם אירוע ANALYTICS.

Elastic Indexer

צורך אירועי RAW, CLEAN ו-ANALYTICS, ומבצע upsert למסמך מאוחד ב-Elasticsearch לפי image_id. המסמך כולל מטא-דאטה, טקסט גולמי, טקסט מנוקה ונתוני אנליטיקה.

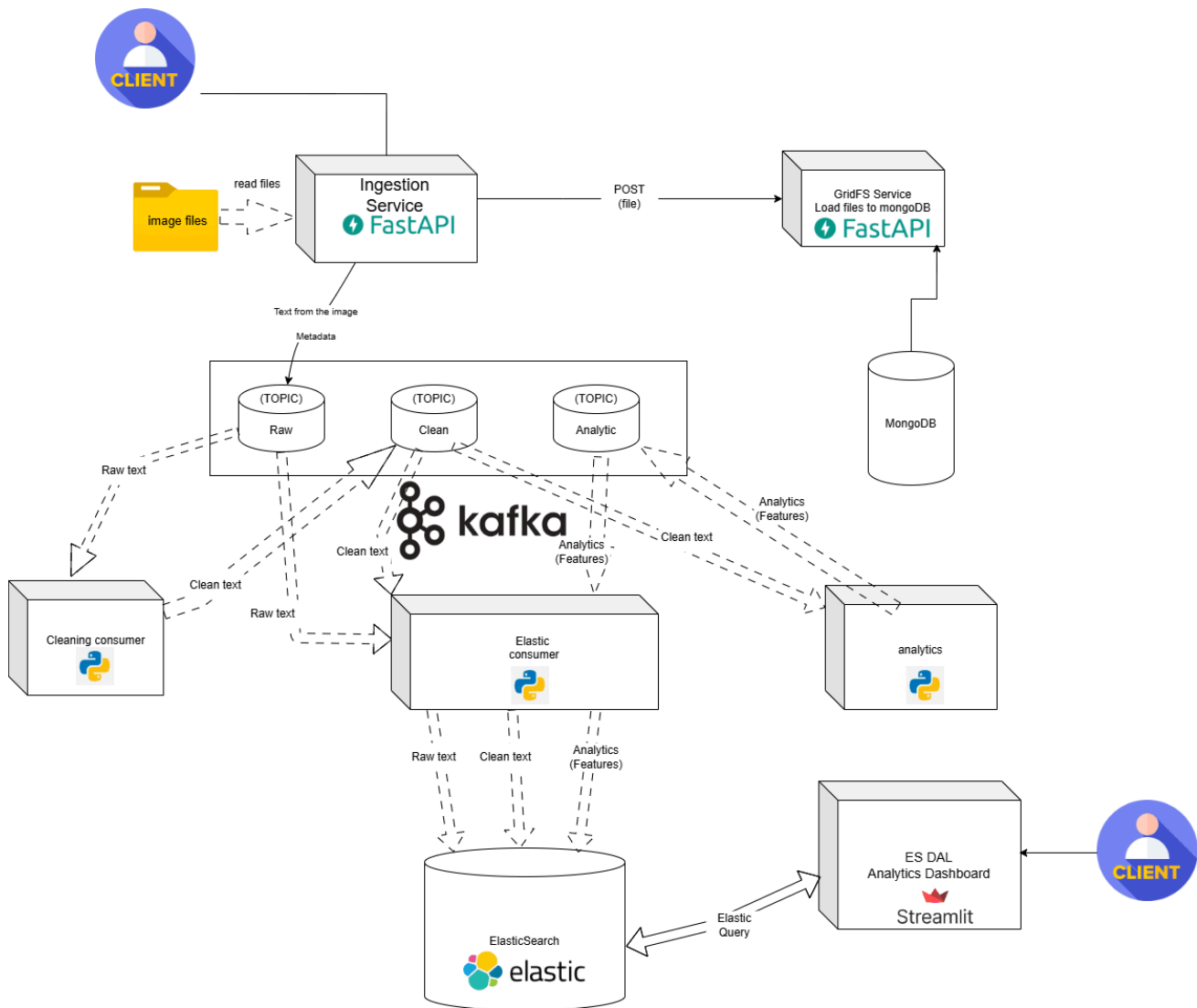
Query API

מספק שכבת REST לביצוע חיפושים ב-Elasticsearch.

Dashboard

ממשק תצוגה המשתמש ב-Query API לצורך הצגת תוצאות.

3. תרשים זרימה לוגי של המערכת



הזרימה היא אירועית (Event-Driven) עד שלב האינדוקס, ולאחר מכן קריאה סינכרונית דרך API.

4. שלד מחלקות לפי שירות

להלן רשימת המחלקות הנדרשות בכל שירות, עם הסבר קצר על אחריות כל אחת.

Ingestion Service

IngestionConfig

טוען משתני סביבה הקשורים ל-Kafka ולנתיב תיקיית התמונות הלוקאלית בתוך הפרויקט.

OCREngine

אחראי על חילוץ הטקסט מתוך תמונה.

MetadataExtractor

אחראי על חילוץ מטא-דאטה בסיסי מתוך קובץ התמונה (כגון גודל, ממדים ופורמט) ועל יצירת image_id ייחודי.

MongoLoaderClient

שולח בקשת POST ל-MongoDB Loader להעברת הקובץ הבינארי בלבד.

KafkaPublisher

מפרסם אירוע RAW ל-Kafka הכולל image_id, raw_text ו-metadata.

IngestionOrchestrator

סורק את תיקיית התמונות המקומית בפרויקט, מפעיל OCR, מחלץ מטא-דאטה, שולח את הקובץ ל-MongoDB Loader ומפרסם אירוע RAW.

GridFS Service

GridFSConfig

טוען משתני סביבה הקשורים ל-MongoDB.

GridFSStorage

אחראי על שמירת הקובץ הבינארי ב-MongoDB באמצעות GridFS.

GridFSOrchestrator

מקבל בקשות POST מה-Ingestion ושומר את הקובץ במסד הנתונים.

Clean Service

CleanConfig

טוען משתני סביבה ל-Kafka.

KafkaConsumer

צורך אירועים מסוג RAW.

TextCleaner

מנקה סימני פיסוק ותווים מיותרים מהטקסט.

KafkaPublisher

מפרסם אירוע CLEAN.

CleanOrchestrator

מתאם בין הצריכה, הניקוי והפרסום.

Analytics Service

AnalyticsConfig

טוען משתני סביבה ל-Kafka.

KafkaConsumer

צורך אירועים מסוג CLEAN.

TextAnalyzer

מחשב מדדים על הטקסט.

KafkaPublisher

מפרסם אירוע ANALYTICS.

AnalyticsOrchestrator

מתאם את זרימת החישוב והפרסום.

Elastic Indexer

IndexerConfig

טוען תצורת Kafka ו-Elasticsearch.

KafkaConsumer

צורך אירועים מכל ה-Topics הרלוונטיים.

ElasticsearchClient

מבצע חיבור ועדכון למסמך באינדקס.

IndexOrchestrator

מתאם בין האירועים ומבצע upsert לפי image_id.

Query API

APIConfig

טוען תצורת Elasticsearch.

SearchService

מבצע שאילתות חיפוש ומחזיר תוצאות.

Routes / Controllers

מקבלים בקשות HTTP ומעבירים אותן ל-SearchService.

Dashboard

DashboardApp

ממשק משתמש המבצע קריאות ל-Query API ומציג תוצאות.

5. Logging – ארכיטקטורה והטמעה

בכל שירות במערכת חייב להתקיים מנגנון Logging עקבי.

העיקרון הארכיטקטוני:

- לכל שירות יש **Logger אחד** שמוגדר ב-main.py.
- ה-Logger מועבר (Dependency Injection) לכל מחלקה דרך ה-constructor.
- אין להשתמש ב-print().

מבנה מומלץ:

main.py

- יוצר Logger עם שם השירות (לדוגמה: "ingestion-service").

- מגדיר רמת לוג מתוך משתנה סביבה LOG_LEVEL.
- מעביר את ה-logger לכל הרכיבים בעת יצירתם.

כל מחלקה

- מקבלת logger ב-constructor.
- משתמשת בו לרישום פעולות משמעותיות (INFO) ושגיאות (ERROR).

דוגמה ארכיטקטונית:

- Orchestrator מדווח על תחילת וסיום תהליך.
- Consumer מדווח על קבלת אירוע.
- Publisher מדווח על פרסום הודעה.
- רכיב לוגיקה מדווח על שגיאות עיבוד.

המטרה היא לאפשר מעקב אחרי image_id לאורך כל ה-Pipeline.

6. פירוט מחלקות – Constructor ומתודות

להלן פירוט תמציתי של כל מחלקה: מה היא מקבלת ב-constructor, מה אחריותה, ואילו מתודות עיקריות עליה לכלול.

Ingestion Service

IngestionConfig

Constructor: ללא פרמטרים; טוען bootstrap_servers, IMAGE_DIRECTORY ל-Kafka ו-MONGO_LOADER_URL.
אחריות: ריכוז תצורה לשירות, כולל נתיב התיקיה הלוקאלית בפרויקט שממנה נקראות התמונות.
מתודות: validate() – בדיקת קיום משתנים.

OCREngine

Constructor: logger.
אחריות: חילוץ טקסט מתוך תמונה.
מתודות: extract_text(image_path) – מחזיר raw_text שחולץ.

MetadataExtractor

Constructor: logger.
אחריות: חילוץ מטא-דאטה בסיסי מתוך קובץ תמונה ויצירת מזהה ייחודי.
מתודות:
extract_metadata(image_path) – מחזיר אובייקט מטא-דאטה (גודל קובץ, ממדים, פורמט וכו').
generate_image_id(image_path) – מייצר image_id ייחודי לקובץ.

MongoLoaderClient

. Constructor: mongo_loader_url, logger
אחריות: שליחת הקובץ הבינארי בלבד ל-MongoDB Loader דרך HTTP.
מתודות: (send(file_path, image_id) – שולח POST ומקבל אישור שמירה.

KafkaPublisher

. Constructor: bootstrap_servers, topic_name, logger
אחריות: פרסום אירוע RAW ל-Kafka.
מתודות: (publish(event) – שולח הודעה ל-Topic.

IngestionOrchestrator

. Constructor: config, ocr_engine, metadata_extractor, mongo_client, publisher, logger
אחריות:
סריקת תיקיית התמונות הלוקאלית בפרויקט (IMAGE_DIRECTORY),
יצירת image_id לכל קובץ,
חילוץ מטא-דאטה,
הפעלת OCR,
שליחת הקובץ הבינארי ל-MongoDB Loader,
ופריסום אירוע RAW הכולל image_id, raw_text ו-metadate.
מתודות:
(process_image(path) – מטפל בתמונה בודדת מקצה לקצה.
(run) – סורק את התיקייה ומפעיל עיבוד על כלל הקבצים.

MongoDB Loader Service

MongoLoaderConfig

. Constructor: MONGO_URI טוען מתוך הסביבה.
אחריות: ריכוז תצורת מסד הנתונים.
מתודות: validate().

GridFSStorage

. Constructor: mongo_uri, logger
אחריות: שמירת הקובץ הבינארי בלבד ב-MongoDB באמצעות GridFS.
מתודות: (save(file_stream, image_id) – שומר קובץ לפי מזהה ומחזיר אישור.

MongoLoaderOrchestrator

. Constructor: storage, logger
אחריות: קבלת בקשות POST מה-Ingestion ושמירת הקובץ במסד הנתונים.
מתודות: (handle_upload(request); run).

Clean Service

CleanConfig

.Kafka: Constructor: טוען תצורת
אחריות: תצורה בלבד.
מתודות: validate().

KafkaConsumer

.Constructor: bootstrap_servers, topic_name, group_id, logger
אחריות: צריכת אירועים מ-Kafka.
מתודות: start(callback) – מאזין ומעביר הודעות לעיבוד.

TextCleaner

.Constructor: logger
אחריות: ניקוי סימני פיסוק ותווים מיותרים.
מתודות: clean(text) – מחזיר טקסט מנוקה.

KafkaPublisher

.Constructor: bootstrap_servers, topic_name, logger
אחריות: פרסום CLEAN.
מתודות: publish(event).

CleanOrchestrator

.Constructor: consumer, cleaner, publisher, logger
אחריות: תיאום בין צריכה, ניקוי ופרסום.
מתודות: handle_event(event) – מנקה ומפרסם; run() – מפעיל לולאת צריכה.

Analytics Service

AnalyticsConfig

Constructor: טוען תצורת Kafka.
אחריות: ריכוז משתני סביבה.
מתודות: validate().

KafkaConsumer

Constructor: bootstrap_servers, topic_name, group_id, logger
אחריות: צריכת CLEAN.
מתודות: start(callback).

TextAnalyzer

Constructor: logger
אחריות: חישוב מדדים בסיסיים על הטקסט.
מתודות: analyze(text) – מחזיר אובייקט מדדים.

KafkaPublisher

Constructor: bootstrap_servers, topic_name, logger
אחריות: פרסום ANALYTICS.
מתודות: publish(event).

AnalyticsOrchestrator

Constructor: consumer, analyzer, publisher, logger
אחריות: תיאום חישוב ופרסום.
מתודות: run; handle_event(event);

Elastic Indexer

IndexerConfig

Constructor: טוען תצורת Kafka ו-Elasticsearch.
אחריות: ריכוז URI והגדרות אינדקס.
מתודות: validate().

KafkaConsumer

Constructor: bootstrap_servers, topics_list, group_id, logger
אחריות: צריכת RAW / CLEAN / ANALYTICS.
מתודות: start(callback).

ElasticsearchClient

.Constructor: es_uri, index_name, logger
אחריות: חיבור ועדכון מסמך באינדקס.
מתודות: (upsert(document, image_id); search(query)

IndexOrchestrator

.Constructor: consumer, es_client, logger
אחריות: מיזוג אירועים למסמך מאוחד לפי image_id.
מתודות: run; handle_event(event);

Query API

APIConfig

.Constructor: טוען תצורת Elasticsearch
אחריות: ריכוז הגדרות API.
מתודות: validate();

SearchService

.Constructor: es_client, logger
אחריות: ביצוע שאילתות חיפוש והחזרת תוצאות.
מתודות: search(filters) – מחזיר רשימת תוצאות.

Routes / Controllers

.Constructor: search_service
אחריות: קבלת בקשות HTTP והעברתן ללוגיקה.
מתודות: search_endpoint(request) – מחזיר תגובת API.

Dashboard

DashboardApp

.Constructor: api_base_url
אחריות: קריאה ל-Query API והצגת נתונים.
מתודות: render(); fetch_results

7. הנחיות כלליות וסיכום

השלד המוצג במסמך זה מגדיר את הדרישה המינימלית למבנה ארכיטקטוני תקין של המערכת. זהו כיוון עבודה מומלץ וברור, אך אין חובה לממש אותו אחד לאחד כל עוד נשמרים העקרונות המרכזיים: הפרדת אחריות, מבנה מודולרי, זרימת נתונים נכונה בין השירותים.

מותר ואף מומלץ:

- להוסיף מתודות במידת הצורך.
- להרחיב מחלקות קיימות אם הדבר משפר קריאות או תחזוקתיות.
- ליצור מחלקות נוספות כאשר יש בכך ערך ארכיטקטוני ברור.
- לאחד רכיבי תשתית (כגון Publisher/Consumer) אם נעשה בצורה נקייה וניתנת לשימוש חוזר.

הדרישה המרכזית היא לכתוב קוד נקי, ברור וקריא:

- כל מחלקה עם אחריות ברורה.
- ללא לוגיקה מיותרת או כפילויות.
- ללא תלות חבויה או State גלובלי.
- שימוש עקבי ב־Logging.

המבנה במסמך זה נועד להכווין אתכם, לא להגביל אתכם. חשבו כמו מהנדסים: אם יש דרך טובה יותר — נמקו אותה ויישמו אותה בצורה נקייה.

בהצלחה במימוש מבצע "רשת הכתבים".